

Probeklausur

Priority Boarding — Familien & Senioren

Programmierung I

Aufgabe 1 (47.25 Punkte)

Implementiere die Klassen

- **Boardable** (1.5 Punkte),
- **PassengerType** (3.5 Punkte),
- **Adult** (3.5 Punkte),
- **SeatRow** (10.0 Punkte),
- **ChildRow** (5.75 Punkte),
- **PriorityRow** (6.0 Punkte),
- **Airplane** (12.0 Punkte) und
- **ExamTask** (5.0 Punkte)

entsprechend dem Klassendiagramm.

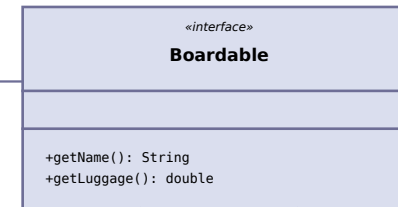
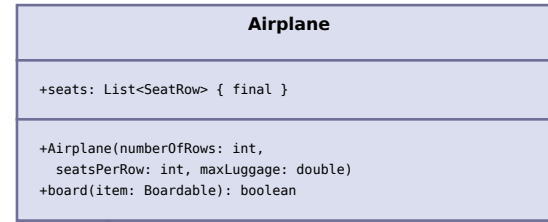
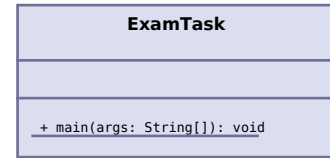
Befolge alle Hinweise bei der Implementierung!

Die Klasse Child ist gegeben und muss nicht implementiert werden.

Glossar

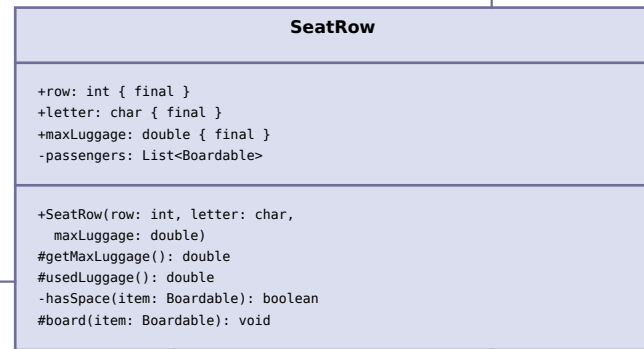
Englisch	Deutsch
adult	Erwachsener
airplane	Flugzeug
available rows	verfuegbare Reihen
board	einsteigen
boardable	einsteigefaeig
child	Kind
child row	Kinderreihe
letter	Buchstabe
luggage	Gepaeck
passenger type	Passagiertyp
priority	Prioritaet
priority row	Prioritaetsreihe
row	Reihe
seat row	Sitzreihe

Klassendiagramm



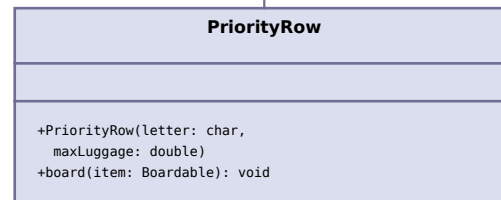
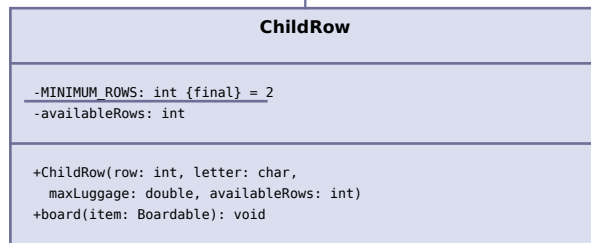
implements

implements



extends

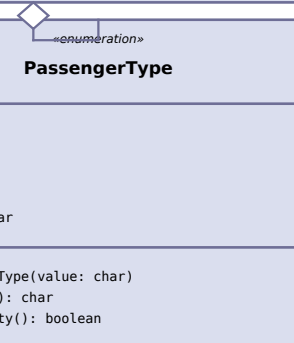
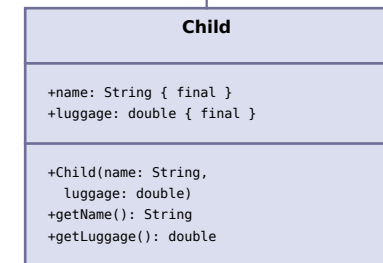
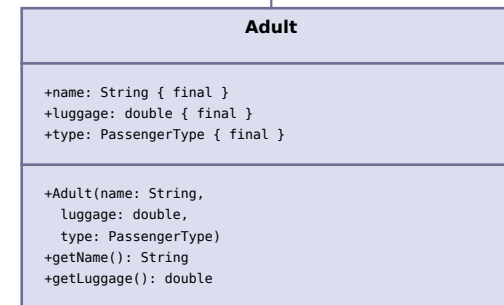
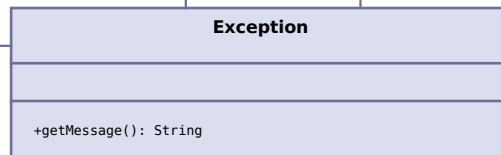
extends



throws

throws

throws



Hinweise zur Klasse Boardable (Einsteigefähig)

- Die Methode **getName** soll den Namen zurückgeben.
- Die Methode **getLuggage** soll das Gepäckgewicht zurückgeben.

Hinweise zur Klasse PassengerType (Passagiertyp)

- Die Methode **getValue** soll den Wert (value) des Passagiertyps zurückgeben.
- Die Methode **hasPriority** soll true zurückgeben, wenn abhängig vom Passagiertyp eine Priorität besteht.

Es besteht Priorität für die Typen FAMILY und SENIOR. Der Typ REGULAR hat keine Priorität.

Hinweise zur Klasse Adult (Erwachsener)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getLuggage** soll das Gepäckgewicht zurückgeben.

Das Gepäckgewicht eines Erwachsenen ist die Hälfte des luggage-Attributs.

Hinweise zur Klasse SeatRow (Sitzreihe)

- **Konstruktor** soll alle Attribute initialisieren.
- Die Methode **getMaxLuggage** soll das maximale Gepäckgewicht einer Sitzreihe berechnen.

Das maximale Gepäckgewicht einer Sitzreihe ist abhängig von maxLuggage.

- Die Methode **usedLuggage** soll das bereits belegte Gepäckgewicht einer Sitzreihe berechnen.

Das belegte Gepäckgewicht ist die Summe des Gepäckgewichts aller Passagiere einer Sitzreihe.

- Die Methode **hasSpace** soll ermitteln, ob der eingehende Passagier in die Sitzreihe passt.

Ein Passagier hat genügend Platz, sobald das Gepäckgewicht des eingehenden Passagiers kleiner ist als das restlich vorhandene Gepäckgewicht einer Sitzreihe.

- Die Methode **board** soll versuchen einen Passagier in der Sitzreihe zu platzieren.

Hat der zu platzierende Passagier genug Platz, soll er der Sitzreihe hinzugefügt werden. Andernfalls soll ein Fehler ausgelöst werden, dass kein Platz mehr vorhanden ist.

Hinweise zur Klasse ChildRow (Kinderreihe)

- **Konstruktor** soll alle Attribute initialisieren.

Die Kinderreihe hat mehr Beinfreiheit und somit 5 Einheiten weniger maximales Gepaeckgewicht.

- Die Methode **board** soll versuchen, einen Passagier in der Kinderreihe zu platzieren.

Nur Kinder (Child) duerfen in der Kinderreihe platziert werden. Ist die Anzahl der verfuegbaren Reihen (availableRows) kleiner als die Mindestanzahl (MINIMUM_ROWS), darf niemand in der Kinderreihe platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgeloeet werden, dass der Familienbereich nicht verfuegbar ist.

Hinweise zur Klasse PriorityRow (Prioritaetsreihe)

- **Konstruktor** soll alle Attribute initialisieren.

Die Prioritaetsreihe soll immer in der untersten Reihe sein (Reihe 1).

- Die Methode **board** soll versuchen, einen Passagier in der Prioritaetsreihe zu platzieren.

Nur Erwachsene (Adult) deren Passagiertyp Prioritaet hat, duerfen in der Prioritaetsreihe platziert werden.

Kann der Passagier nicht platziert werden, soll ein Fehler ausgeloeet werden, dass nur Prioritaetspassagiere erlaubt sind.

Hinweise zur Klasse Airplane (Flugzeug)

- **Konstruktor** soll alle Attribute initialisieren.

Jedes Flugzeug hat mehrere Sitzreihen. Es gibt jedoch immer genau eine Kinderreihe und eine Prioritaetsreihe. Der Konstruktor soll nach der Initialisierung aller Attribute zuerst die Kinderreihe, dann die Prioritaetsreihe und anschliessend die normalen Sitzreihen erzeugen und der internen Liste hinzufuegen.

Die normalen Sitzreihen werden in einem verschachtelten Loop erzeugt: Der aeussere Loop iteriert ueber die Reihen (absteigend), der innere Loop ueber die Sitzbuchstaben (A, B, C, ...).

Die Prioritaetsreihe hat immer die geringste Reihe (Reihe 1) und die Kinderreihe immer die hoechste Reihe. Die Kinderreihe hat 4 verfuegbare Reihen (availableRows).

Beispiel Liste mit numberOfRows = 3 und seatsPerRow = 2:

```
- ChildRow          - Reihe = 5, Buchstabe = A
- PriorityRow        - Reihe = 1, Buchstabe = A
- normale Sitzreihe - Reihe = 4, Buchstabe = A
- normale Sitzreihe - Reihe = 4, Buchstabe = B
- normale Sitzreihe - Reihe = 3, Buchstabe = A
- normale Sitzreihe - Reihe = 3, Buchstabe = B
- normale Sitzreihe - Reihe = 2, Buchstabe = A
- normale Sitzreihe - Reihe = 2, Buchstabe = B
```

- Die Methode **board** soll einen Passagier platzieren und zurueckgeben, ob es funktioniert hat.

Versuche einen Platz fuer den Passagier im Flugzeug zu finden. Wenn ein Platz gefunden wurde, soll auf der Konsole die Reihe und der Buchstabe ausgegeben werden.

Wenn ein Passagier in einer Sitzreihe nicht platziert werden konnte, soll der Grund auf der Konsole ausgegeben werden.

Hinweise zur Klasse ExamTask

Es soll ein Flugzeug mit 4 Reihen, 3 Sitzreihen pro Reihe und einem maximalen Gepaeckgewicht von 90 erstellt werden.

Erstelle ein Kind mit einem Namen deiner Wahl und einem Gepaeckgewicht von 5 und versuche es im Flugzeug zu platzieren.

Iteriere ueber alle Passagiertypen. Erstelle fuer jeden Passagiertyp einen Erwachsenen mit einem Gepaeckgewicht von 70 und versuche ihn im Flugzeug zu platzieren. Verwende den Wert (getValue) des Passagiertyps im Namen des Erwachsenen.

Falls kein Platz gefunden wurde, soll dies zusammen mit dem Wert des Passagiertyps auf der Konsole ausgegeben werden.

Java API

Klasse	Methode	Statisch	Rueckgabotyp
PassengerType	values()	X	PassengerType[]
PassengerType	name()		String
Exception	getMessage()		String
Boolean	valueOf(s: String)	X	Boolean
Boolean	valueOf(b: boolean)	X	Boolean
Double	valueOf(s: String)	X	Double
Double	valueOf(d: double)	X	Double
Integer	valueOf(s: String)	X	Integer
Integer	valueOf(i: int)	X	Integer
String	charAt(index: int)		char
String	length()		int

Java Collections Framework

Klasse	Methode	Statisch	Rueckgabotyp
ArrayList<T>	add(element: T)		boolean
ArrayList<T>	add(index: int, element: T)		void
ArrayList<T>	contains(element: T)		boolean
ArrayList<T>	get(index: int)		T
ArrayList<T>	remove(index: int)		T
ArrayList<T>	remove(element: T)		boolean
ArrayList<T>	size()		int
Collections	sort(list: List<T>)	X	void
Collections	sort(list, c: Comparator<T>)	X	void

Schnittstellen

Klasse	Methode	Statisch	Rueckgabotyp
Comparable<T>	compareTo(o: T)		int
Comparator<T>	compare(o1: T, o2: T)		int